

.4.1 Pointers and the Address Operator

Pointers are variables that contain the memory address for another variable. A pointer is defined like a normal variable, but with an asterisk before the variable name. The type-specifier determines what kind of variable the pointer points to but does not affect the actual pointer.

The address operator causes the memory address for a variable to be returned. It is written with an ampersand sign before the variable name.

When using a pointer, referencing just the pointer such as:

```
int *my_pointer;  
int barny;  
my_pointer=&barny;
```

Causes `my_pointer` to contain the address of `barny`. Now the pointer can be use indirection to reference the variable it points to. Indirection is done by prefixing an asterisk to the pointer variable.

```
*my_pointer=3;
```

This causes the value of `barny` to be 3. Note that the value of `my_pointer` is unchanged.

Pointers offer an additional method for addressing an array. The following array:

```
int my_array[3];
```

Can be addressed normally such as:

```
my_array[2]=3;
```

The same can be accomplished with:

```
*(my_array+2)=3;
```

Note that `my_array` is a pointer *constant*. Its value cannot be modified such as:

```
my_array++; This is illegal.
```

However, if a pointer variable is created such as:

```
int *some_pointer=my_array;
```

Then modifying the pointer will correctly increment the pointer so as to point to the next element in the array.

```
*(some_pointer+1)=3;
```

This will cause the value of `my_array[1]` to be 3. On a system where an `int` takes up two bytes, adding 1 to `some_pointer` did not actually increase it by 1, but by 2 so that it pointed to the next element in the

array.

Functions can also be represented with a pointer. A function pointer is defined in the same way as a function prototype, but the function name is replaced by the pointer name prefixed with an asterisk and encapsulated with parenthesis. Such as:

```
int (*fptr)(int, char);  
fptr=some_function;
```

To call this function:

```
(*fptr)(3, 'A');
```

This is equivalent to:

```
some_function(3, 'A');
```

A structure or union can have a pointer to represent it. Such as:

```
struct some_structure homer;  
struct some_structure *homer_pointer;  
homer_pointer=&homer;
```

This defines homer_pointer to point to the structure homer. Now, when you use the pointer to reference something in the structure, the record selector now becomes -> instead of a period.

```
homer_pointer->an_element=5;
```

This is the same as:

```
homer.an_element=5;
```

The void pointer can represent an unknown pointer type.

```
void *joe;
```

This is a pointer to an undetermined type.

1.4.2 Typecasting

Typecasting allows a variable to act like a variable of another type. The method of typecasting is done by prefixing the variable type enclosed by parenthesis before the variable name. The actual variable is not modified.

Example:

```
float index=3; int loop=(int)index;
```

This causes index to be typecasted to act like an int.